



NGUYÊN LÝ HỆ ĐIỀU HÀNH

Phần 6: Bể tắc

NGUYỄN THỊ HẬU

Khoa Công Nghệ Thông Tin
Đại học Công Nghệ - Đại học quốc gia Hà Nội

Phần 6: Bể tắc

1. Bài toán bể tắc
2. Các điều kiện cần để có bể tắc
3. Đồ thị phân phối tài nguyên
4. Các phương pháp xử lý bể tắc
5. Khôi phục khi bể tắc đã xảy ra

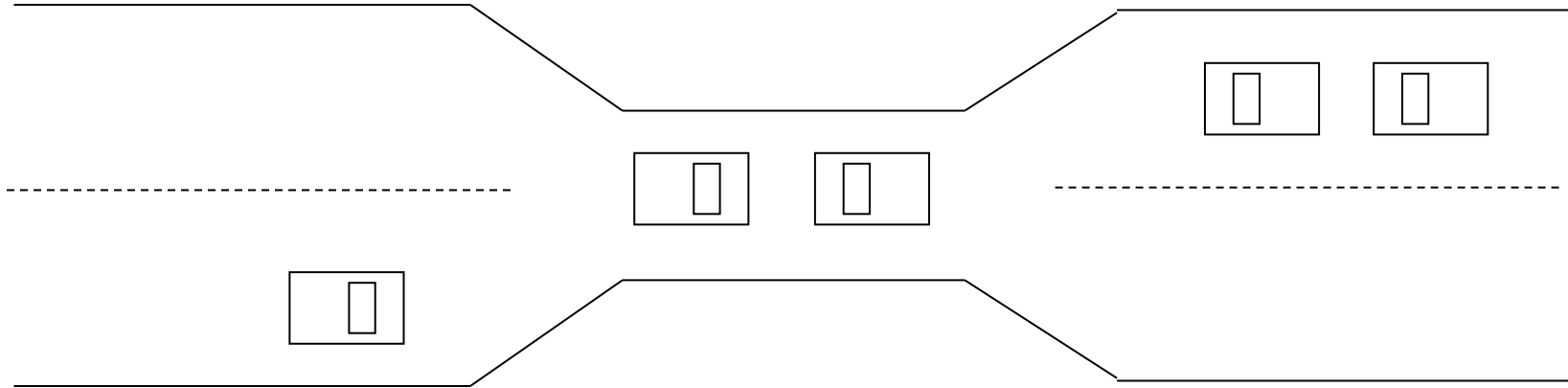
Phần 6: Bể tắc

6.1 Bài toán bể tắc

Bài toán bế tắc

- Một tập hợp các tiến trình, trong đó mỗi tiến trình giữ một tài nguyên và đợi một tài nguyên đang bị chiếm giữ bởi một tiến trình khác trong tập hợp này
- Ví dụ:
 - Hai semaphore A và B được khởi tạo bằng 1
 - P_0 : wait (A); wait(B)
 - P_1 : wait (B); wait(A)

Bài toán qua cầu



- Chỉ cho phép một làn xe qua cầu
- Khi tắc xảy ra, một số xe phải lùi lại
- Có khả năng xảy ra nạn đói

6.2 Các điều kiện cần để có bế tắc

Bế tắc xảy ra khi 4 điều kiện sau đồng thời diễn ra

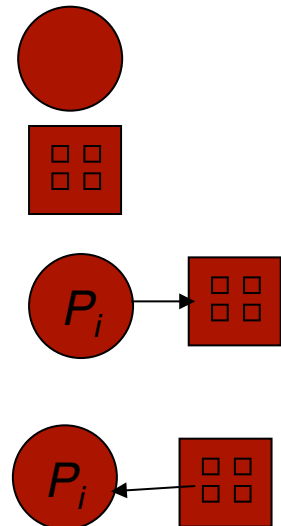
- **Loại trừ lẫn nhau:** mỗi thời điểm, chỉ một tiến trình được sử dụng tài nguyên
- **Giữ và đợi:** tiến trình chiếm giữ một tài nguyên và chờ một tài nguyên khác đang chiếm giữ bởi tiến trình khác
- **Không dừng:** tài nguyên chỉ được giải phóng bởi tiến trình chiếm giữ nó sau khi kết thúc nhiệm vụ
- **Chờ đợi vòng tròn:** có một tập các tiến trình $\{P_0, P_1, \dots, P_n\}$ trong đó tiến trình P_i đang đợi một tài nguyên bị chiếm giữ bởi tiến trình $P_{(i+1)\%(n+1)}$

Phần 6: Bế tắc

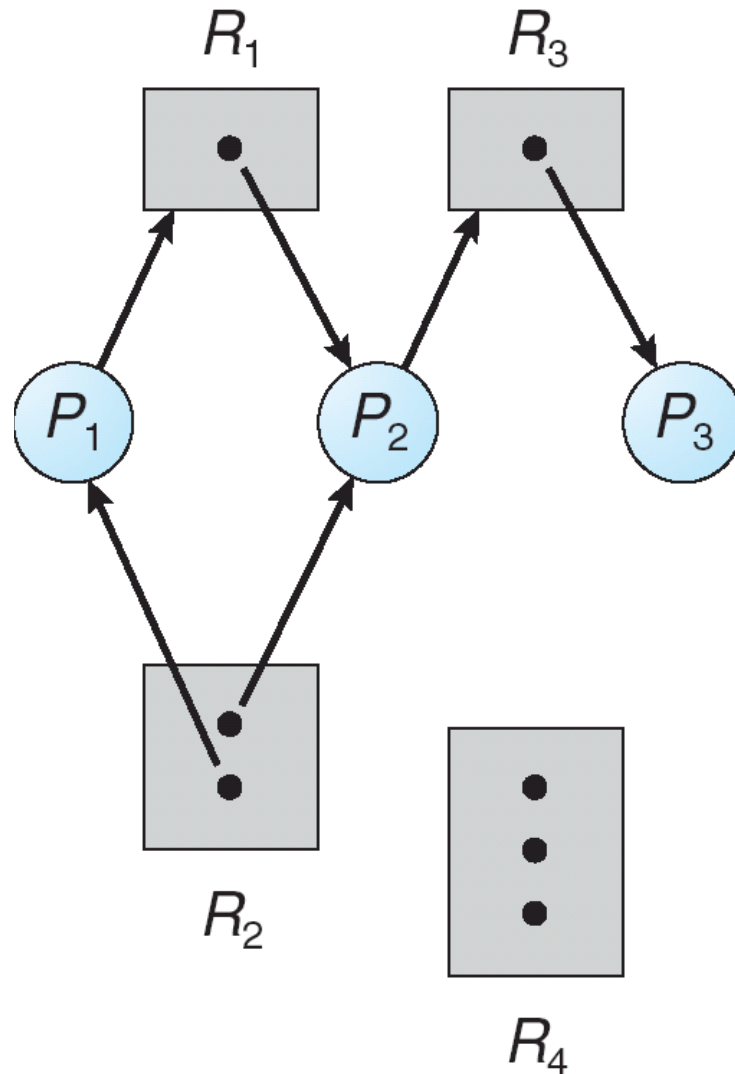
6.3 Đồ thị phân phối tài nguyên

Đồ thị phân phối tài nguyên

- V : tập các đỉnh
 - $P = \{P_1, P_2, \dots, P_n\}$ tập các tiến trình trong hệ thống
 - $R = \{R_1, R_2, \dots, R_m\}$ tập các nguồn tài nguyên trong hệ thống
- E : tập các cạnh
 - Cạnh yêu cầu: $P_i \rightarrow R_j$
 - Cạnh cấp phát: $R_j \rightarrow P_i$
- Tiến trình
- Tài nguyên gồm 4 đơn vị
- P_i yêu cầu một đơn vị của tài nguyên R_j
- P_i chiếm giữ một đơn vị của tài nguyên R_j

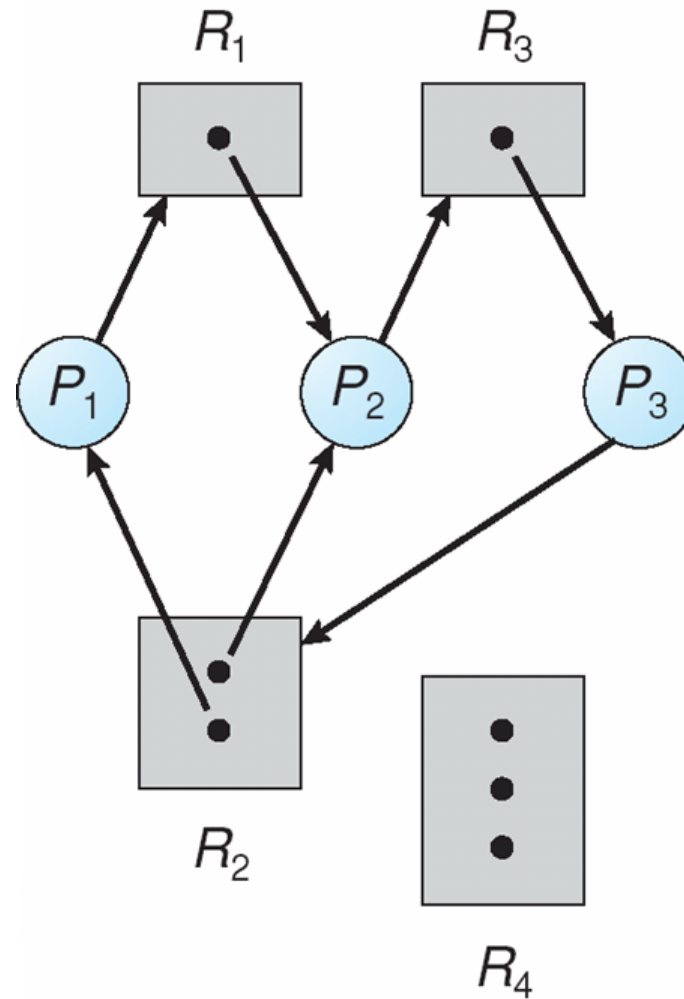


Ví dụ đồ thị phân phối tài nguyên



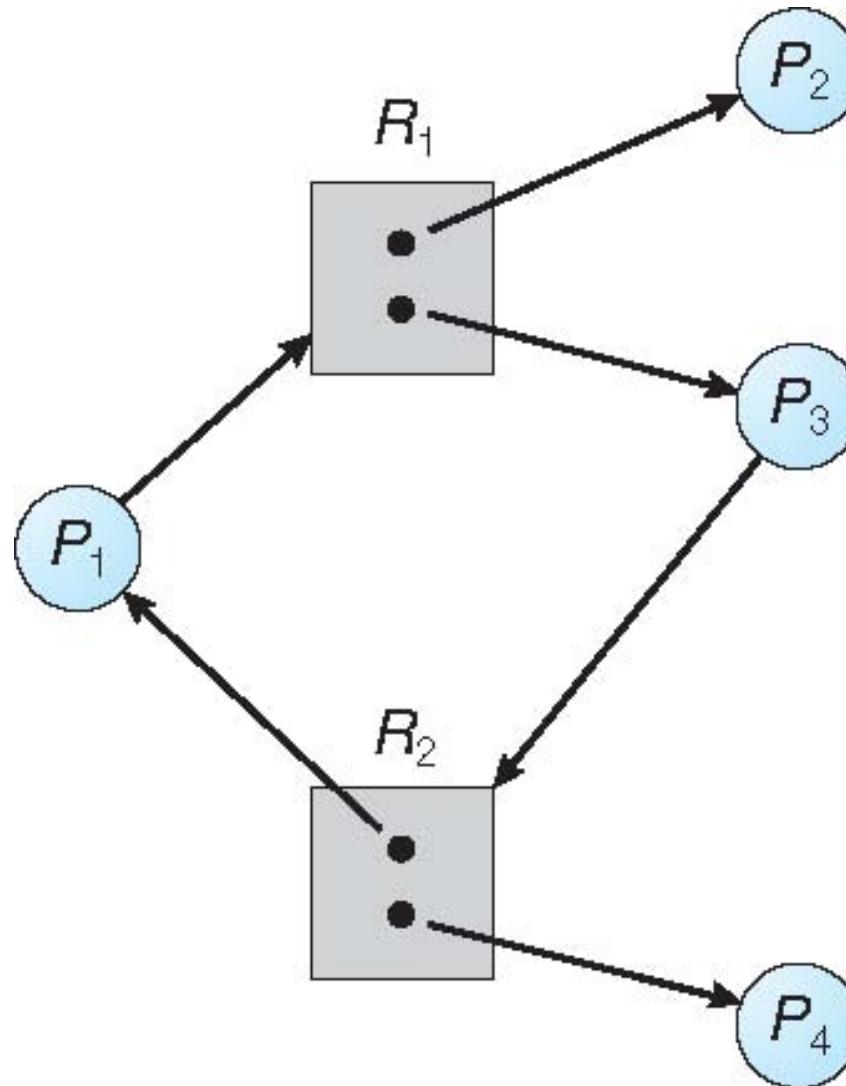
Ví dụ đồ thị phân phối tài nguyên có bế tắc

10



Ví dụ đồ thị phân phối tài nguyên có chờ vòng tròn nhưng không bế tắc

11



Phần 6: Bể tắc

6.4 Các phương pháp xử lý khi có bể tắc

Các phương pháp xử lý khi có bế tắc

- Sử dụng các phương thức phòng hoặc tránh bế tắc nhằm đảm bảo hệ thống không bao giờ vào trạng thái bế tắc
- Cho phép hệ thống vào trạng thái bế tắc, phát hiện và khôi phục lại
- Bỏ qua và giả định rằng bế tắc không bao giờ xảy ra (hầu hết các HĐH lựa chọn cách thức này, bao gồm cả Linux và Windows)

C1: Phòng tránh bế tắc bằng cách loại bỏ một trong bốn điều kiện cho phép bế tắc xảy ra

14

■ Loại trừ lẫn nhau:

- Không cần đối với nguồn tài nguyên cho phép chia sẻ, nhưng cần đối với nguồn tài nguyên không cho phép chia sẻ

■ Giữ và chờ:

- Cho phép tiến trình yêu cầu một tài nguyên khi nó không chiếm giữ tài nguyên nào cả, hoặc
- Yêu cầu các tiến trình thông báo các yêu cầu về tài nguyên và cấp phát tất cả các nguồn tài nguyên cho tiến trình trước thực thi
- **Nhược điểm:** Mức sử dụng tài nguyên thấp và có thể xảy ra nạn đói

C1: Phòng tránh bế tắc bằng cách loại bỏ một trong bốn điều kiện cho phép bế tắc xảy ra 15

▪ Không dùng:

▪ Cách 1:

- Nếu tiến trình đang chiếm giữ một số tài nguyên và yêu cầu một tài nguyên khác mà nó không thể có ngay thì tất cả tài nguyên nó đang chiếm giữ được giải phóng
- Các tài nguyên được giải phóng được thêm vào danh sách tài nguyên mà tiến trình đang chờ

▪ Cách 2:

- Nếu các nguồn tài nguyên được yêu cầu bởi tiến trình P đang bị chiếm giữ bởi một tiến trình Q và Q đang chờ một số tài nguyên khác thì dừng Q lại, cấp phát các tài nguyên được yêu cầu cho tiến trình P
 - Nếu các nguồn tài nguyên P cần không có sẵn hoặc không bị chiếm giữ bởi một tiến trình đang chờ khác, thì P phải chờ. Khi đó, một số tài nguyên do P chiếm giữ sẽ được giải phóng nếu tiến trình khác yêu cầu
 - Tiến trình khởi động lại chỉ khi nó lấy được tất cả tài nguyên cũ và tài nguyên mới yêu cầu
- ## ▪ Chờ đợi vòng tròn: Đánh số thứ tự cho các tài nguyên, và mỗi tiến trình yêu cầu tài nguyên theo thứ tự tăng dần

C2: Phòng tránh bế tắc bằng cách thêm thông tin về các yêu cầu tài nguyên

16

- Yêu cầu mỗi tiến trình khai báo số lượng đơn vị tối đa mỗi loại tài nguyên mà nó cần
- Sử dụng thuật toán kiểm tra trạng thái cấp phát tài nguyên để đảm bảo không có vòng tròn chờ đợi
- Trạng thái cấp phát tài nguyên là số lượng tài nguyên có sẵn, số lượng tài nguyên đang sử dụng và số lượng tài nguyên được yêu cầu bởi các tiến trình

- Khi tiến trình yêu cầu một tài nguyên có sẵn, hệ thống phải quyết định liệu việc cấp phát tài nguyên đó cho phép hệ thống ở trạng thái an toàn
- Hệ thống ở **trạng thái an toàn** nếu có một chuỗi các tiến trình trong hệ thống $\langle P_1, P_2, \dots, P_n \rangle$ sao cho với mỗi P_i , tài nguyên mà P_i có thể yêu cầu là các tài nguyên sẵn sàng hoặc các tài nguyên chiếm giữ bởi P_j với $j < i$
- Nghĩa là:
 - Nếu tài nguyên mà P_i không có sẵn ngay thì P_i có thể chờ đến khi P_j xong
 - Khi P_j xong thì P_i lấy được các tài nguyên cần thiết, thực thi, trả lại tài nguyên rồi kết thúc

Ví dụ trạng thái an toàn

18

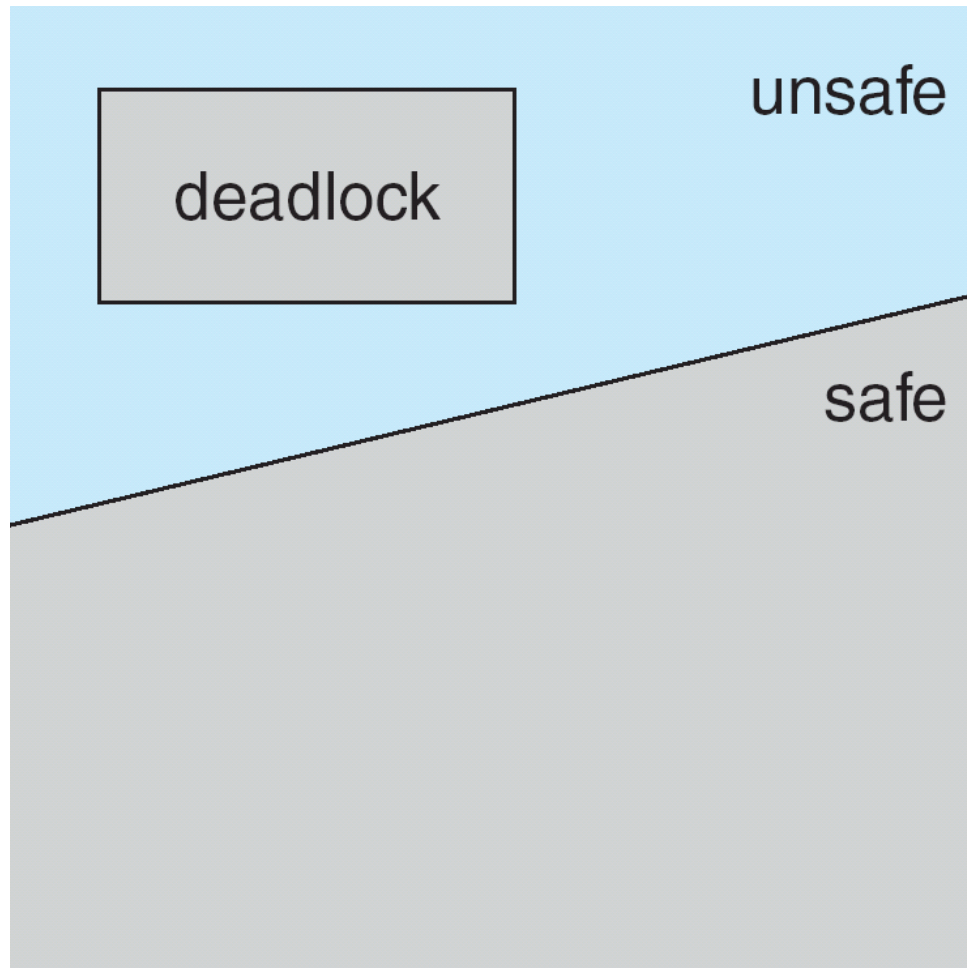
Hệ thống có 12 ổ đĩa từ, ở thời điểm t_0 yêu cầu về sử dụng ổ đĩa từ như sau:

	<i>Cần tối đa</i>	<i>Đang chiếm giữ</i>
P_0	10	5
P_1	4	2
P_2	9	2

Trạng thái của hệ thống ở thời điểm t_0 là an toàn vì chuỗi $\langle P_1, P_0, P_2 \rangle$ thoả mãn điều kiện an toàn

Trạng thái an toàn, không an toàn, bế tắc

19



Thuật toán phòng tránh bế tắc

20

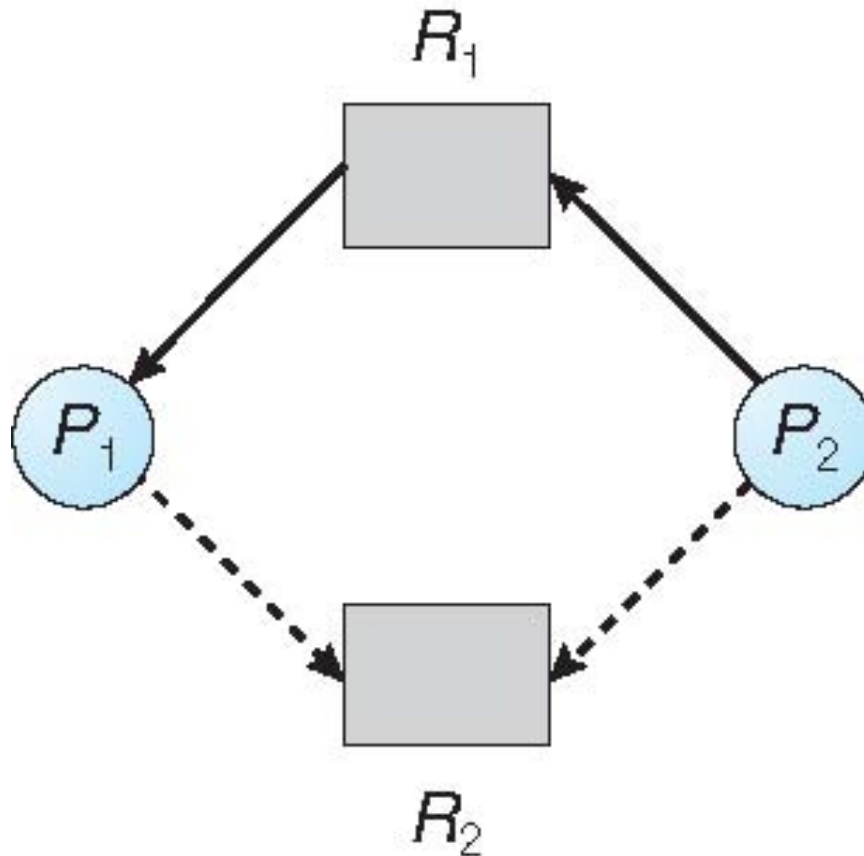
- Nếu mỗi tài nguyên chỉ có một đơn vị:
 - Sử dụng đồ thị phân phối tài nguyên

- Nếu mỗi tài nguyên có nhiều đơn vị
 - Sử dụng thuật toán banker

Sử dụng đồ thị phân phối tài nguyên

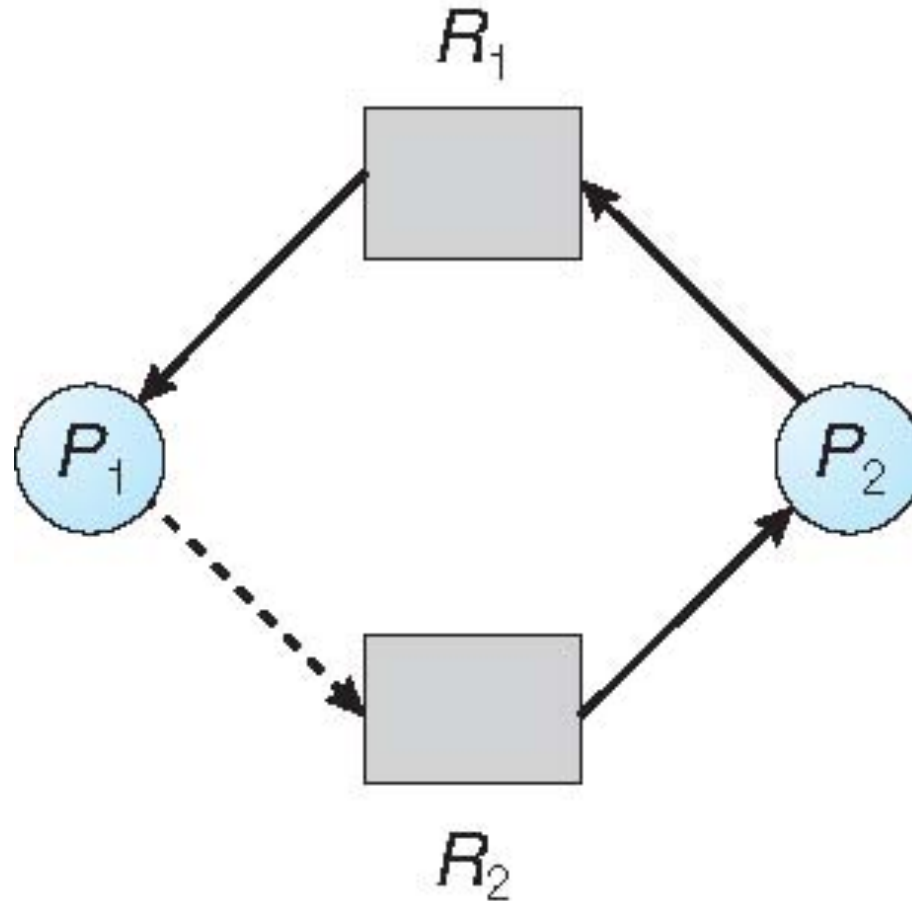
- **Cạnh khai báo** $P_i \rightarrow R_j$: tiến trình P_i có thể sẽ yêu cầu tài nguyên R_j , được biểu diễn bằng đường nét đứt
- **Cạnh khai báo** chuyển thành **cạnh yêu cầu** khi tiến trình yêu cầu tài nguyên
- **Cạnh yêu cầu** chuyển thành **cạnh cấp phát** khi tài nguyên được cấp phát cho tiến trình
- Khi tài nguyên được giải phóng bởi một tiến trình, **cạnh cấp phát** chuyển thành **cạnh khai báo**
- Phải khai báo tài nguyên cho hệ thống

Sử dụng đồ thị phân phối tài nguyên



Sử dụng đồ thị phân phối tài nguyên – trạng thái không an toàn

23



Nếu cạnh khai báo $P_1 \text{ --- } R_2$ chuyển thành cạnh yêu cầu thì sẽ xảy ra bế tắc

Sử dụng thuật toán banker (số an toàn hay k)

- Mỗi tài nguyên có thể có nhiều đơn vị
- Mỗi tiến trình phải khai báo số lượng tài nguyên tối đa sẽ sử dụng
- Khi tiến trình yêu cầu tài nguyên, nó có thể phải chờ
- Khi tiến trình lấy tất cả các tài nguyên cần thiết, nó phải trả lại sau một khoảng thời gian có hạn

Sử dụng thuật toán banker

n = số tiến trình, m = số nguồn tài nguyên

- **Available:** mảng gồm m phần tử. Nếu $\text{Available}[j] = k$, thì có k đơn vị tài nguyên R_j có sẵn
- **Max:** Ma trận $n \times m$. Nếu $\text{Max}[i,j] = k$, thì tiến trình P_i được yêu cầu nhiều nhất k đơn vị tài nguyên R_j
- **Allocation:** Ma trận $n \times m$. Nếu $\text{Allocation}[i,j] = k$ thì P_i đang chiếm giữ k đơn vị của R_j
- **Need:** Ma trận $n \times m$. Nếu $\text{Need}[i,j] = k$, thì P_i có thể cần thêm k đơn vị của R_j để hoàn thành nhiệm vụ

$$\text{Need}[i,j] = \text{Max}[i,j] - \text{Allocation}[i,j]$$

Thuật toán an toàn

1. Work và Finish là hai mảng có độ dài m và n. Khởi tạo:

$$Work = Available$$

$$Finish[i] = false \text{ với } i = 0, 1, \dots, n-1$$

2. Tìm i sao cho thoả mãn

(a) $Finish[i] = false$

(b) $Need_i \leq Work$

Nếu không có i thoả mãn, chuyển sang bước 4

3. $Work = Work + Allocation_i$

$$Finish[i] = true$$

Chuyển sang bước 2

4. Nếu $Finish[i] == true$ với tất cả i, thì hệ thống ở trạng thái an toàn

Thuật toán kiểm tra yêu cầu cấp phát tài nguyên cho P_i

27

Request = mảng yêu cầu tài nguyên của P_i . Nếu $Request_i[j] = k$ thì P_i muốn k đơn vị của tài nguyên R_j

1. Nếu $Request_i \leq Need_i$ thì sang bước 2. Nếu không thì thông báo lỗi vì tiến trình sử dụng quá số lượng tài nguyên tối đa đã khai báo
2. Nếu $Request_i \leq Available$, thì sang bước 3. Nếu không P_i phải chờ vì tài nguyên không sẵn sàng
3. Giả định rằng cấp phát tài nguyên cho P_i thông qua câu lệnh sau:

$$Available = Available - Request_i$$

$$Allocation_i = Allocation_i + Request_i$$

$$Need_i = Need_i - Request_i$$

- Nếu an toàn thì tài nguyên được cấp phát cho P_i
- Nếu không an toàn $\Rightarrow P_i$ phải chờ, trạng thái cấp phát tài nguyên trước được hồi phục

Ví dụ thuật toán banker

■ 5 tiến trình $P_0 P_1 P_2 P_3 P_4$

■ 3 nguồn tài nguyên:

A (10 đơn vị), B (5 đơn vị), and C (7 đơn vị)

tại thời điểm t_0 :

	<i>Allocation</i>	<i>Max</i>	<i>Available</i>	<i><u>Need</u> = Max – Allocation</i>
	<i>A B C</i>	<i>A B C</i>	<i>A B C</i>	<i>A B C</i>
P_0	0 1 0	7 5 3	3 3 2	7 4 3
P_1	2 0 0	3 2 2		1 2 2
P_2	3 0 2	9 0 2		6 0 0
P_3	2 1 1	2 2 2		0 1 1
P_4	0 0 2	4 3 3		4 3 1

■ Hệ thống ở trạng thái an toàn không ?

■ Có vì chuỗi $\langle P_1, P_3, P_4, P_2, P_0 \rangle$ thoả mãn điều kiện an toàn

Ví dụ: P_1 yêu cầu (1,0,2)

29

- Kiểm tra liệu $\text{Request}_1 \leq \text{Available}$ (tức là $(1,0,2) \leq (3,3,2) \Rightarrow \text{true}$)
- Giả sử yêu cầu của P_1 được chấp nhận thì trạng thái mới của hệ thống là:

	<u>Allocation</u>	<u>Need</u>	<u>Available</u>
	<i>A B C</i>	<i>A B C</i>	<i>A B C</i>
P_0	0 1 0	7 4 3	2 3 0
P_1	3 0 2	0 2 0	
P_2	3 0 2	6 0 0	
P_3	2 1 1	0 1 1	
P_4	0 0 2	4 3 1	

- Thực thi thuật toán an toàn thì chuỗi $\langle P_1, P_3, P_4, P_0, P_2 \rangle$ thoả mãn điều kiện an toàn
- Liệu yêu cầu (3,3,0) bởi P_4 được chấp nhận ?
- Liệu yêu cầu (0,2,0) bởi P_0 được chấp nhận ?

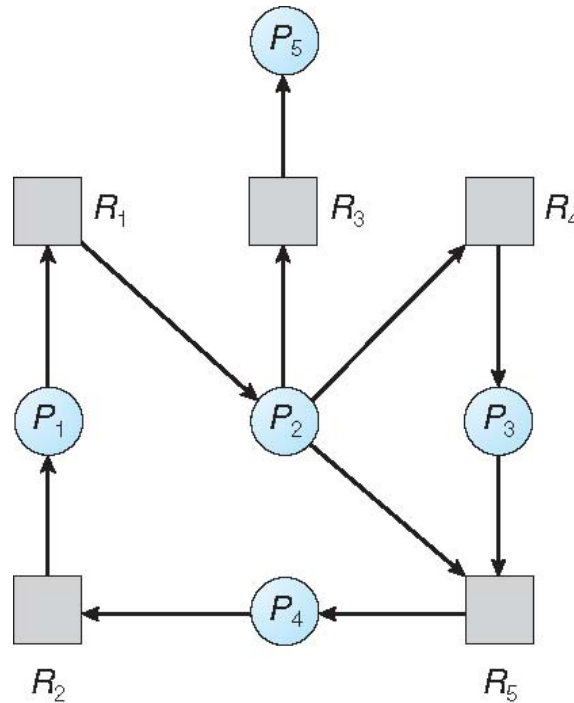
Phần 6: Bể tắc

6.5 Khôi phục khi bể tắc xảy ra

Sử dụng đồ thị wait - for

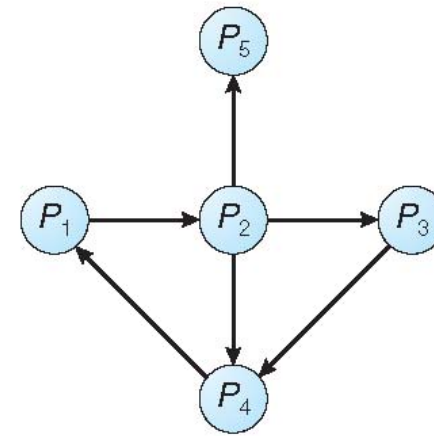
31

- Mỗi tài nguyên chỉ có một đơn vị
- Đỉnh đồ thị là các tiến trình
- Cạnh $P_i \rightarrow P_j$: P_i đang chờ P_j
- Định kỳ sử dụng thuật toán để kiểm tra liệu có chu trình trong đồ thị. Nếu có thì có bế tắc



(a)

Đồ thị cấp phát tài nguyên



(b)

Đồ thị wait-for tương ứng

Thuật toán nhận diện *lệ tài* *(có tài hay không)*

32

- Mỗi nguồn tài nguyên có thể có nhiều hơn một đơn vị

n = số tiến trình, m = số nguồn tài nguyên

- **Available:** mảng gồm m phần tử. Nếu $\text{Available}[j] = k$, thì có k đơn vị tài nguyên R_j có sẵn
- **Allocation:** Ma trận $n \times m$. Nếu $\text{Allocation}[i,j] = k$ thì P_i đang chiếm giữ k đơn vị của R_j
- **Request:** Ma trận $n \times m$. Nếu $\text{Request}[i,j] = k$, thì P_i đang yêu cầu thêm k đơn vị của R_j

1. Work và Finish là hai mảng có độ dài m và n. Khởi tạo:
 - (a) $Work = Available$
 - (b) Với $i = 0, 1, \dots, n-1$, nếu $Allocation_i \neq 0$ thì $Finish[i] = false$, ngược lại $Finish[i] = true$
2. Tìm i sao cho thoả mãn
 - (a) $Finish[i] = false$
 - (b) $Request_i \leq Work$Nếu không có i thoả mãn, chuyển sang bước 4
3. $Work = Work + Allocation_i$
 $Finish[i] = true$
Chuyển sang bước 2
4. Nếu $Finish[i] == false$ với một số $0 \leq i \leq n-1$, thì hệ thống ở trạng thái bế tắc.
Và nếu $Finish[i] == false$ thì P_i bị bế tắc

Độ phức tạp của thuật toán $O(m.n^2)$

Ví dụ: Thuật toán nhận diện

34

- 5 tiến trình $P_0 P_1 P_2 P_3 P_4$
- 3 nguồn tài nguyên: A (7 đơn vị), B (2 đơn vị), and C (6 đơn vị)
- Tại thời điểm t_0 , trạng thái của hệ thống là:

	<u>Allocation</u>	<u>Request</u>	<u>Available</u>
	$A \ B \ C$	$A \ B \ C$	$A \ B \ C$
P_0	0 1 0	0 0 0	0 0 0
P_1	2 0 0	2 0 2	
P_2	3 0 3	0 0 0	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	

- Chuỗi $\langle P_0, P_2, P_3, P_1, P_4 \rangle$ cho phép $Finish[i] = \text{true}$ với tất cả $0 \leq i \leq n-1$

Ví dụ: Thuật toán nhận diện

35

- P_2 yêu cầu thêm một đơn vị tài nguyên C

Request

$A \ B \ C$

$P_0 \ 0 \ 0 \ 0$

$P_1 \ 2 \ 0 \ 2$

$P_2 \ 0 \ 0 \ 1$

$P_3 \ 1 \ 0 \ 0$

$P_4 \ 0 \ 0 \ 2$

- Hệ thống đang ở trạng thái nào ?

Tần suất sử dụng thuật toán nhận diện

36

Phụ thuộc vào

- Tần suất bế tắc xuất hiện
- Số lượng tiến trình liên quan khi bế tắc xảy ra

Khôi phục khi bế tắc xảy ra: Dừng tiến trình

37

- Dừng tất cả tiến trình bị bế tắc, hoặc
- Dừng từng tiến trình cho đến khi hết bế tắc
- Thứ tự dừng tiến trình:
 - Thứ tự ưu tiên của tiến trình
 - Thời gian tiến trình đã chạy, thời gian còn lại
 - Tài nguyên tiến trình đã sử dụng
 - Tài nguyên tiến trình cần thêm
 - Số lượng tiến trình sẽ bị dừng
 - Tiến trình tương tác hay xử lý theo lô

Khôi phục khi bế tắc xảy ra: Dừng tài nguyên

38

- **Lựa chọn tài nguyên và tiến trình bị dừng:** giảm thiểu chi phí
- **Quay lại:** quay lại trạng thái an toàn nào đó, khởi động lại tiến trình từ trạng thái đó
- **Nạn đói:** Có thể một tiến trình nào đó luôn bị chọn để dừng lại (do nhân tố chi phí)

Bài tập

- Viết chương trình minh họa thuật toán banker, thuật toán nhận diện bế tắc